

TOPIC-5: GREEDY ALGORITHMS

Q1. Maximum Number of Coins You Can Have

Aim:

To implement a greedy algorithm to find the maximum number of coins that you can obtain from $3n$ piles of coins.

Algorithm:

Step 1: Sort the piles of coins in ascending order.

Step 2: Initialize two pointers:

- Start pointer for Bob's smallest piles.
- End pointer for Alice's largest piles.

Step 3: In every round:

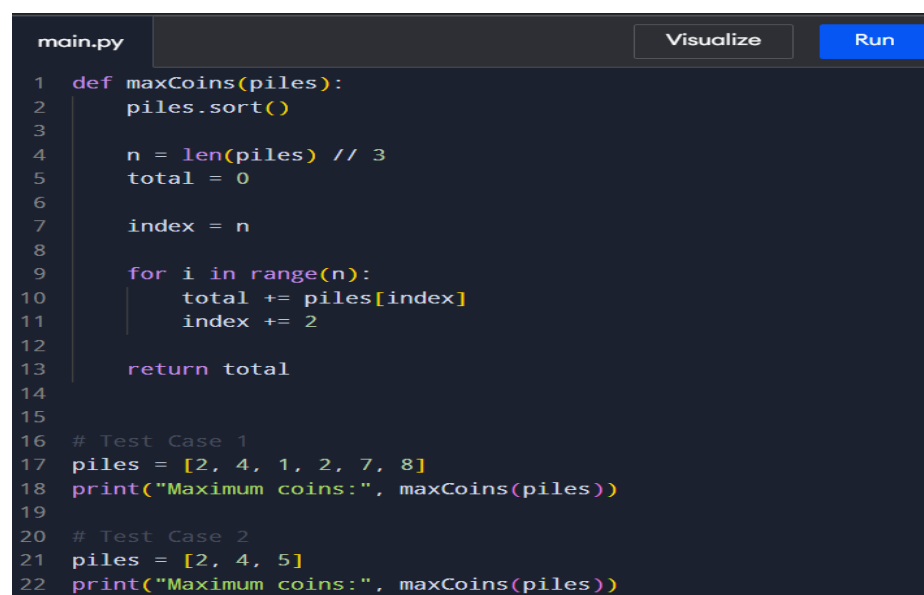
- Alice takes the largest pile.
- You take the second-largest pile.
- Bob takes the smallest pile.

Step 4: Add your selected pile to the total.

Step 5: Repeat until all piles are selected.

Step 6: Return the maximum number of coins.

Python Code:

A screenshot of a code editor window titled 'main.py'. The editor has a dark background with light-colored text. On the right side of the editor, there are two buttons: 'Visualize' and 'Run'. The code is a Python function 'maxCoins' that takes a list of piles and returns the maximum number of coins. It sorts the piles in ascending order and then iterates through them, taking the largest pile (index), the second-largest pile (index+1), and the smallest pile (index-1) in each round. The code includes two test cases: one with 6 piles [2, 4, 1, 2, 7, 8] and one with 3 piles [2, 4, 5].

```
1 def maxCoins(piles):
2     piles.sort()
3
4     n = len(piles) // 3
5     total = 0
6
7     index = n
8
9     for i in range(n):
10        total += piles[index]
11        index += 2
12
13    return total
14
15
16 # Test Case 1
17 piles = [2, 4, 1, 2, 7, 8]
18 print("Maximum coins:", maxCoins(piles))
19
20 # Test Case 2
21 piles = [2, 4, 5]
22 print("Maximum coins:", maxCoins(piles))
```

Output:

```
Output

Maximum coins: 9
Maximum coins: 4

=== Code Execution Successful ===
```

Time Complexity – $O(n \log n)$ | Space Complexity – $O(n)$

Q2. Minimum Coins to Add

Aim:

To find the minimum number of coins that need to be added so that every value from 1 to target can be formed.

Algorithm:

Step 1: Sort the given coins.

Step 2: Initialize reachable = 0, representing all values from 1 to reachable that can be formed.

Step 3: If the next coin is less than or equal to reachable + 1, include it.

Step 4: Otherwise, add a new coin with value reachable + 1.

Step 5: Update the reachable range.

Step 6: Repeat until reachable $\geq$ target.

Step 7: Return the number of added coins.

Python Code:

main.py

VisualizeRun

```
1 def minimumAddedCoins(coins, target):
2     coins.sort()
3
4     reachable = 0
5     count = 0
6     i = 0
7
8     while reachable < target:
9
10         if i < len(coins) and coins[i] <= reachable + 1:
11             reachable += coins[i]
12             i += 1
13         else:
14             reachable += reachable + 1
15             count += 1
16
17     return count
18
19
20 # Test Case 1
21 coins = [1, 4, 10]
22 target = 19
23
24 print("Minimum coins to add:",
25       minimumAddedCoins(coins, target))
26
27
28 # Test Case 2
29 coins = [1, 4, 10, 5, 7, 19]
```

Output:

Output

```
Minimum coins to add: 2
Minimum coins to add: 1

=== Code Execution Successful ===
```

Time Complexity – $O(n \log n)$ | Space Complexity – $O(1)$

Q3. Minimum Maximum Working Time

Aim:

To assign jobs among workers such that the maximum working time of any worker is minimized.

Algorithm:

Step 1: Sort the jobs in descending order.

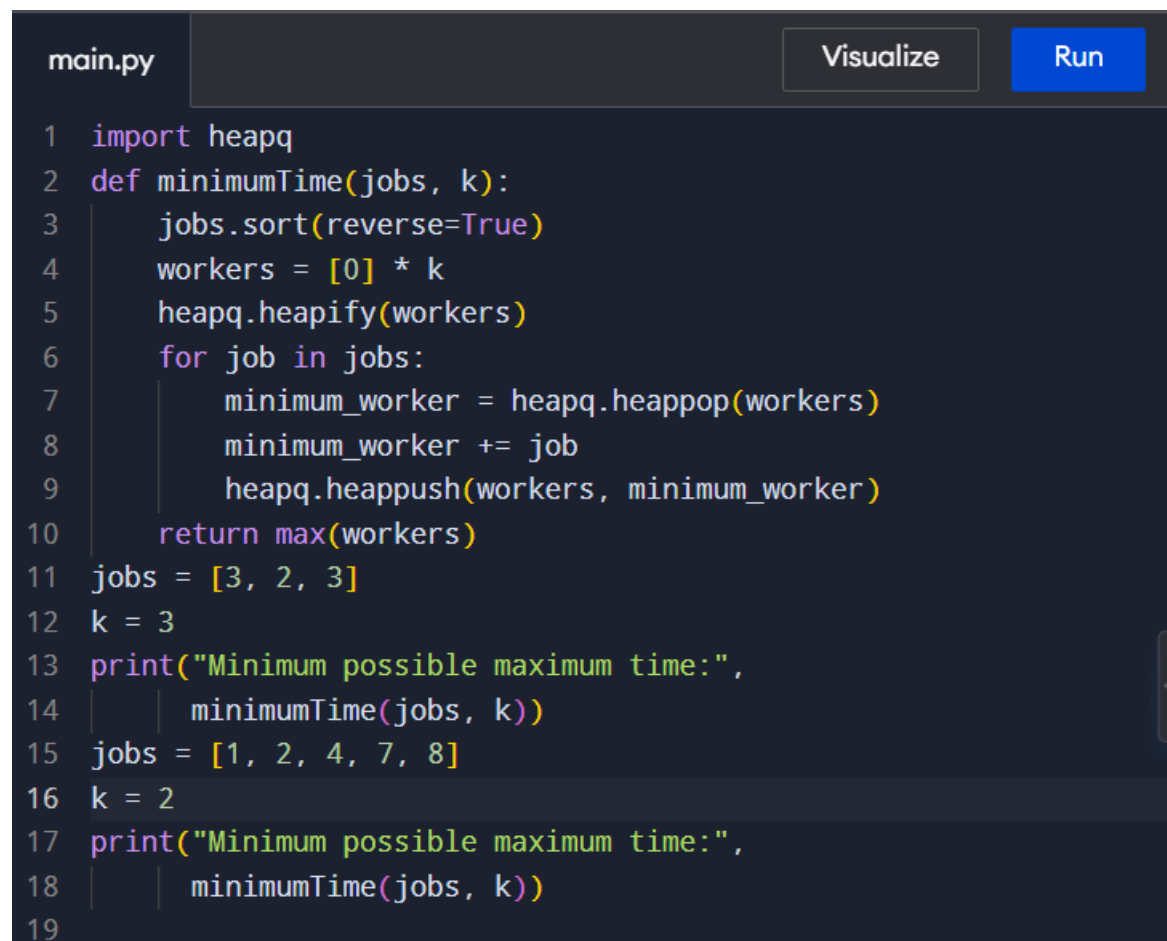
Step 2: Assign each job to the worker with the minimum current workload.

Step 3: Update the workload of that worker.

Step 4: Repeat until all jobs are assigned.

Step 5: Return the maximum workload among all workers.

Python Code:



```
main.py Visualize Run
1 import heapq
2 def minimumTime(jobs, k):
3     jobs.sort(reverse=True)
4     workers = [0] * k
5     heapq.heapify(workers)
6     for job in jobs:
7         minimum_worker = heapq.heappop(workers)
8         minimum_worker += job
9         heapq.heappush(workers, minimum_worker)
10    return max(workers)
11 jobs = [3, 2, 3]
12 k = 3
13 print("Minimum possible maximum time:",
14       minimumTime(jobs, k))
15 jobs = [1, 2, 4, 7, 8]
16 k = 2
17 print("Minimum possible maximum time:",
18       minimumTime(jobs, k))
19
```

Output:

Output

```
Minimum possible maximum time: 3  
Minimum possible maximum time: 11  
  
=== Code Execution Successful ===
```

Time Complexity – $O(n \log n + nm)$ | Space Complexity – $O(m)$

Q4. Maximum Profit in Job Scheduling

Aim:

To find the maximum profit by selecting non-overlapping jobs.

Algorithm:

Step 1: Combine start time, end time, and profit information.

Step 2: Sort jobs according to their end times.

Step 3: For each job, find the latest non-overlapping job.

Step 4: Compare:

- Profit including the current job.
- Profit excluding the current job.

Step 5: Store the maximum profit.

Step 6: Return the maximum profit.

Python Code:

main.py Visualize Run

```
1 from bisect import bisect_right
2 def jobScheduling(startTime, endTime, profit):
3     jobs = sorted(zip(startTime, endTime, profit),
4                   key=lambda x: x[1])
5     ends = [job[1] for job in jobs]
6     dp = [0] * (len(jobs) + 1)
7     for i in range(1, len(jobs) + 1):
8         start, end, money = jobs[i - 1]
9         index = bisect_right(ends, start, 0, i - 1)
10        dp[i] = max(dp[i - 1], dp[index] + money)
11    return dp[-1]
12 startTime = [1, 2, 3, 3]
13 endTime = [3, 4, 5, 6]
14 profit = [50, 10, 40, 70]
15 print("Maximum profit:",
16       jobScheduling(startTime, endTime, profit))
17 startTime = [1, 2, 3, 4, 6]
18 endTime = [3, 5, 10, 6, 9]
19 profit = [20, 20, 100, 70, 60]
20 print("Maximum profit:",
21       jobScheduling(startTime, endTime, profit))
```

Output:

Output

```
Maximum profit: 120
Maximum profit: 150

=== Code Execution Successful ===
```

Time Complexity – $O(n \log n)$ | Space Complexity – $O(n)$

Q5. Dijkstra's Algorithm Using Adjacency Matrix

Aim:

To implement Dijkstra's Algorithm to find the shortest path from a source vertex to all other vertices using an adjacency matrix.

Algorithm:

Step 1: Initialize the source distance as 0.

Step 2: Set all other distances to Infinity.

Step 3: Select the unvisited vertex with the smallest distance.

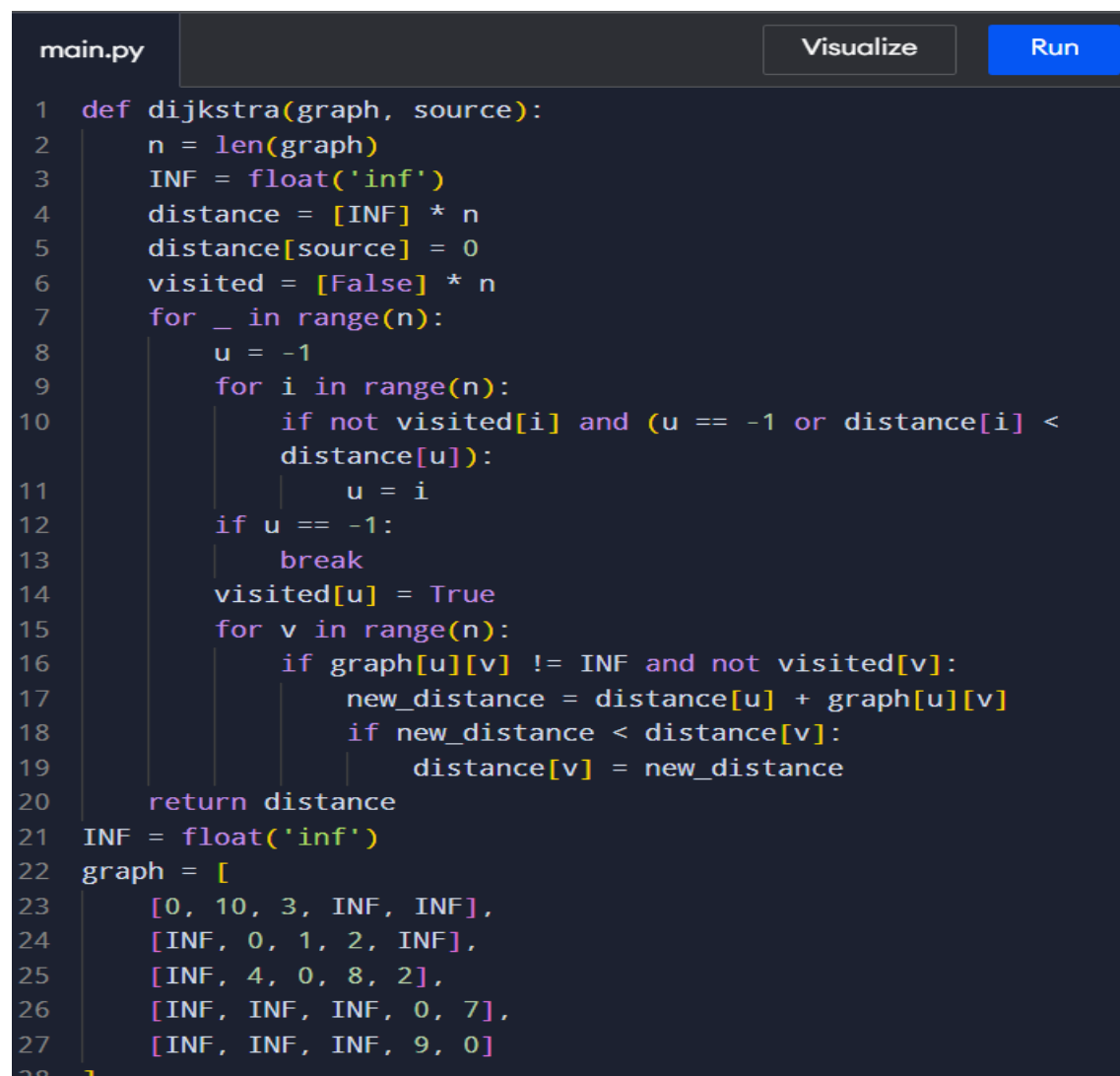
Step 4: Mark the selected vertex as visited.

Step 5: Update the distances of its adjacent vertices.

Step 6: Repeat until all vertices are visited.

Step 7: Return the shortest distances.

Python Code:

A screenshot of a Python IDE window titled 'main.py'. The window has a dark background with light-colored text. At the top right, there are two buttons: 'Visualize' and 'Run'. The code is a Python function 'dijkstra' that takes a graph and a source vertex as input. It initializes a distance array with infinity for all vertices except the source, which is set to 0. It then enters a loop to find the shortest path by selecting the unvisited vertex with the smallest distance, marking it as visited, and updating the distances of its neighbors. The graph is represented as an adjacency list. The code is as follows:

```
1 def dijkstra(graph, source):
2     n = len(graph)
3     INF = float('inf')
4     distance = [INF] * n
5     distance[source] = 0
6     visited = [False] * n
7     for _ in range(n):
8         u = -1
9         for i in range(n):
10            if not visited[i] and (u == -1 or distance[i] <
11                distance[u]):
12                u = i
13        if u == -1:
14            break
15        visited[u] = True
16        for v in range(n):
17            if graph[u][v] != INF and not visited[v]:
18                new_distance = distance[u] + graph[u][v]
19                if new_distance < distance[v]:
20                    distance[v] = new_distance
21    return distance
22 INF = float('inf')
23 graph = [
24     [0, 10, 3, INF, INF],
25     [INF, 0, 1, 2, INF],
26     [INF, 4, 0, 8, 2],
27     [INF, INF, INF, 0, 7],
28     [INF, INF, INF, 9, 0]
```

```
29 print("Shortest distances:", dijkstra(graph, 0))
30 graph2 = [
31     [0, 5, INF, 10],
32     [INF, 0, 3, INF],
33     [INF, INF, 0, 1],
34     [INF, INF, INF, 0]
35 ]
36 print("Shortest distances:", dijkstra(graph2, 0))
```

Output:

Output

```
Shortest distances: [0, 7, 3, 9, 5]
Shortest distances: [0, 5, 8, 9]
```

```
=== Code Execution Successful ===
```

Time Complexity – $O(V^2)$ | Space Complexity – $O(V^2)$

Q6. Dijkstra's Algorithm Using Edge List

Aim:

To find the shortest path from a source vertex to a target vertex using Dijkstra's Algorithm.

Algorithm:

Step 1: Convert the edge list into an adjacency list.

Step 2: Initialize the source distance as 0.

Step 3: Use a priority queue to select the vertex with minimum distance.

Step 4: Update distances of adjacent vertices.

Step 5: Continue until the target is reached.

Step 6: Return the shortest distance.

Python Code:

main.py

Visualize

Run

```
1 import heapq
2 def shortestPath(n, edges, source, target):
3     graph = [[] for _ in range(n)]
4     for u, v, w in edges:
5         graph[u].append((v, w))
6     distance = [float('inf')] * n
7     distance[source] = 0
8     pq = [(0, source)]
9     while pq:
10         dist, u = heapq.heappop(pq)
11         if dist > distance[u]:
12             continue
13         for v, weight in graph[u]:
14             new_dist = dist + weight
15             if new_dist < distance[v]:
16                 distance[v] = new_dist
17                 heapq.heappush(pq, (new_dist, v))
18     return distance[target]
19 n = 6
20 edges = [
21     (0, 1, 7), (0, 2, 9), (0, 5, 14),
22     (1, 2, 10), (1, 3, 15),
23     (2, 3, 11), (2, 5, 2),
24     (3, 4, 6), (4, 5, 9)
25 ]
26 print("Shortest path:", shortestPath(n, edges, 0, 4))
27 n = 5
28 edges = [
29     (0, 1, 10), (0, 4, 3),
30     (1, 2, 2), (1, 4, 4),
31     (2, 3, 9), (3, 2, 7),
32     (4, 1, 1), (4, 2, 8),
33     (4, 3, 2)
34 ]
35 print("Shortest path:", shortestPath(n, edges, 0, 3))
```

Output:

```
Output

Shortest path: 26
Shortest path: 5

=== Code Execution Successful ===
```

Time Complexity – $O(E \log V)$ | Space Complexity – $O(V + E)$

Q7. Huffman Coding

Aim:

To construct a Huffman Tree and generate Huffman Codes for each character.

Algorithm:

Step 1: Create a node for every character and its frequency.

Step 2: Insert all nodes into a priority queue.

Step 3: Remove the two nodes with minimum frequency.

Step 4: Create a new node with their combined frequency.

Step 5: Insert the new node back into the queue.

Step 6: Repeat until only one node remains.

Step 7: Traverse the tree:

- Left edge = 0
- Right edge = 1

Step 8: Generate codes for all characters.

Python Code:

main.py

VisualizeRun

```
1 import heapq
2 class Node:
3     def __init__(self, char, freq):
4         self.char = char
5         self.freq = freq
6         self.left = None
7         self.right = None
8     def __lt__(self, other):
9         return self.freq < other.freq
10 def huffmanCodes(characters, frequencies):
11     heap = []
12     for char, freq in zip(characters, frequencies):
13         heapq.heappush(heap, Node(char, freq))
14     while len(heap) > 1:
15         left = heapq.heappop(heap)
16         right = heapq.heappop(heap)
17         new_node = Node(None, left.freq + right.freq)
18         new_node.left = left
19         new_node.right = right
20         heapq.heappush(heap, new_node)
21     root = heap[0]
22     codes = {}
23     def generateCodes(node, code=""):
24         if node is None:
25             return
26         if node.char is not None:
27             codes[node.char] = code
28             return
29         generateCodes(node.left, code + "0")
30         generateCodes(node.right, code + "1")
31     generateCodes(root)
32     return codes
33 characters = ['a', 'b', 'c', 'd']
34 frequencies = [5, 9, 12, 13]
35 codes = huffmanCodes(characters, frequencies)
```

Output:

Output

```
{'a': '00', 'b': '01', 'c': '10', 'd': '11'}
```

```
=== Code Execution Successful ===
```

Time Complexity – $O(n \log n)$ | Space Complexity – $O(n)$

Q8. Huffman Decoding

Aim:

To decode a Huffman encoded string and obtain the original message.

Algorithm:

Step 1: Construct the Huffman Tree using character frequencies.

Step 2: Start from the root of the tree.

Step 3: Read each bit from the encoded string.

Step 4: Move left for 0.

Step 5: Move right for 1.

Step 6: When a leaf node is reached, add its character to the result.

Step 7: Return to the root and continue.

Step 8: Return the decoded message.

Python Code:

main.py

VisualizeRun

```
1 import heapq
2 class Node:
3     def __init__(self, char, freq):
4         self.char = char
5         self.freq = freq
6         self.left = None
7         self.right = None
8     def __lt__(self, other):
9         return self.freq < other.freq
10 def buildHuffmanTree(characters, frequencies):
11     heap = []
12     for char, freq in zip(characters, frequencies):
13         heapq.heappush(heap, Node(char, freq))
14     while len(heap) > 1:
15         left = heapq.heappop(heap)
16         right = heapq.heappop(heap)
17         parent = Node(None, left.freq + right.freq)
18         parent.left = left
19         parent.right = right
20         heapq.heappush(heap, parent)
21     return heap[0]
22 def decode(root, encoded_string):
23     result = ""
24     current = root
25     for bit in encoded_string:
26         if bit == '0':
27             current = current.left
28         else:
29             current = current.right
30             if current.left is None and current.right is None:
31                 result += current.char
32                 current = root
33     return result
34 characters = ['a', 'b', 'c', 'd']
35 frequencies = [5, 9, 12, 13]
36 root = buildHuffmanTree(characters, frequencies)
37 encoded_string = "1101100111110"
38 print("Decoded message:",
39       decode(root, encoded_string))
```

Time Complexity – $O(L)$ | Space Complexity – $O(n)$

Q9. Maximum Weight Loading Using Greedy Approach

Aim:

To load the maximum possible weight into a container by selecting heavier items first.

Algorithm:

Step 1: Sort the weights in descending order.

Step 2: Initialize current weight as 0.

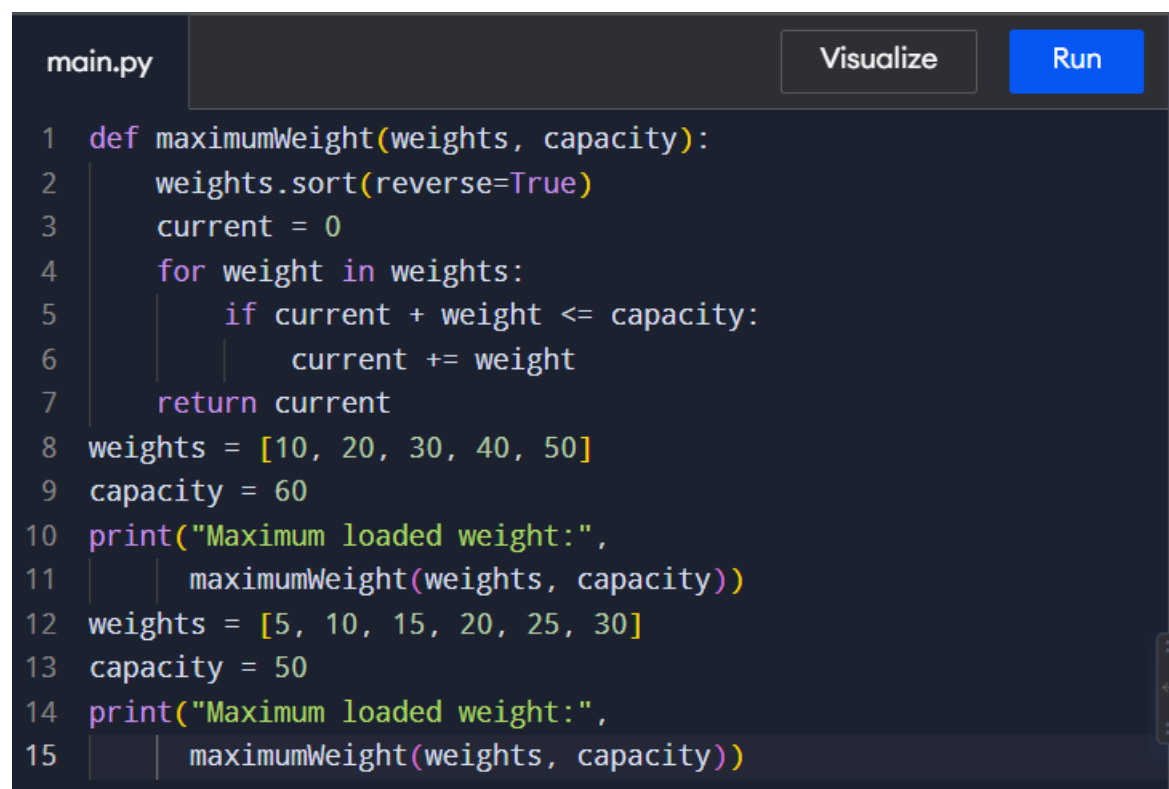
Step 3: Select the heaviest item.

Step 4: Add it only if the total does not exceed capacity.

Step 5: Repeat for all items.

Step 6: Return the maximum loaded weight.

Python Code:

A screenshot of a Python code editor with a dark theme. The editor has a tab labeled 'main.py' at the top left. To the right of the tab are two buttons: 'Visualize' and 'Run'. The code is written in Python and implements a greedy algorithm for maximum weight loading. It defines a function 'maximumWeight' that takes 'weights' and 'capacity' as arguments. The function sorts the weights in descending order, initializes a 'current' weight to 0, and then iterates through the sorted weights, adding each weight to 'current' if the total does not exceed the capacity. Finally, it returns the 'current' weight. Below the function definition, there are two test cases: one with weights [10, 20, 30, 40, 50] and capacity 60, and another with weights [5, 10, 15, 20, 25, 30] and capacity 50. Both test cases print the maximum loaded weight.

```
1 def maximumWeight(weights, capacity):
2     weights.sort(reverse=True)
3     current = 0
4     for weight in weights:
5         if current + weight <= capacity:
6             current += weight
7     return current
8 weights = [10, 20, 30, 40, 50]
9 capacity = 60
10 print("Maximum loaded weight:",
11       maximumWeight(weights, capacity))
12 weights = [5, 10, 15, 20, 25, 30]
13 capacity = 50
14 print("Maximum loaded weight:",
15       maximumWeight(weights, capacity))
```

Output:

Output

Maximum loaded weight: 60

Maximum loaded weight: 50

=== Code Execution Successful ===

Time Complexity – $O(n \log n)$ | Space Complexity – $O(n)$

Q10. Minimum Number of Containers

Aim:

To determine the minimum number of containers required to load all items using a greedy approach.

Algorithm:

Step 1: Sort the weights in descending order.

Step 2: Start with an empty container.

Step 3: Add items to the current container while capacity allows.

Step 4: If the next item cannot fit, open a new container.

Step 5: Repeat until all items are loaded.

Step 6: Return the number of containers.

Python Code:

```
main.py Visualize Run
1 def minimumContainers(weights, capacity):
2     weights.sort(reverse=True)
3     containers = 0
4     current = 0
5     for weight in weights:
6         if current + weight <= capacity:
7             current += weight
8         else:
9             containers += 1
10            current = weight
11    if current > 0:
12        containers += 1
13    return containers
14 weights = [5, 10, 15, 20, 25, 30, 35]
15 print("Containers required:",
16       minimumContainers(weights, 50))
17 weights = [10, 20, 30, 40, 50, 60, 70, 80]
18 print("Containers required:",
19       minimumContainers(weights, 100))
```

Output:

```
Output

Containers required: 4
Containers required: 5

=== Code Execution Successful ===
```

Time Complexity – $O(n \log n)$ | Space Complexity – $O(n)$

Q11. Kruskal's Algorithm for Minimum Spanning Tree**Aim:**

To implement Kruskal's Algorithm to find the Minimum Spanning Tree and its total weight.

Algorithm:

Step 1: Sort all edges according to weight.

Step 2: Initialize each vertex as a separate set.

Step 3: Select the edge with minimum weight.

Step 4: Check whether adding the edge creates a cycle.

Step 5: If no cycle is created, add the edge to the MST.

Step 6: Repeat until $n - 1$ edges are selected.

Step 7: Calculate the total weight.

Python Code:

main.py

Visualize

Run

```
1 def find(parent, x):
2     if parent[x] != x:
3         parent[x] = find(parent, parent[x])
4     return parent[x]
5 def union(parent, rank, x, y):
6     rootX = find(parent, x)
7     rootY = find(parent, y)
8     if rootX != rootY:
9         if rank[rootX] < rank[rootY]:
10             parent[rootX] = rootY
11         elif rank[rootX] > rank[rootY]:
12             parent[rootY] = rootX
13         else:
14             parent[rootY] = rootX
15             rank[rootX] += 1
16 def kruskal(n, edges):
17     edges.sort(key=lambda x: x[2])
18     parent = list(range(n))
19     rank = [0] * n
20     mst = []
21     total = 0
22     for u, v, weight in edges:
23         if find(parent, u) != find(parent, v):
24             mst.append((u, v, weight))
25             total += weight
26             union(parent, rank, u, v)
27     return mst, total
28 edges = [
```

```
29     (0, 1, 10),
30     (0, 2, 6),
31     (0, 3, 5),
32     (1, 3, 15),
33     (2, 3, 4)
34 ]
35 mst, total = kruskal(4, edges)
36 print("Edges in MST:", mst)
37 print("Total weight:", total)
```

Output:

```
Output

Edges in MST: [(2, 3, 4), (0, 3, 5), (0, 1, 10)]
Total weight: 19

=== Code Execution Successful ===
```

Time Complexity – $O(E \log E)$ | Space Complexity – $O(V + E)$

Q12. Verify Whether an MST is Unique

Aim:

To verify whether a given Minimum Spanning Tree is unique and find another possible MST if it is not unique.

Algorithm:

Step 1: Find the minimum MST weight using Kruskal's Algorithm.

Step 2: For each edge in the MST, temporarily remove that edge.

Step 3: Find another MST.

Step 4: Compare its weight with the original MST.

Step 5: If another MST has the same weight, the MST is not unique.

Step 6: Otherwise, the MST is unique.

Python Code:

```
main.py Visualize Run

1 def kruskal_mst(n, edges, banned_edge=None):
2     edges = sorted(edges, key=lambda x: x[2])
3     parent = list(range(n))
4     def find(x):
5         if parent[x] != x:
6             parent[x] = find(parent[x])
7         return parent[x]
8     mst = []
9     total = 0
10    for edge in edges:
11        if edge == banned_edge:
12            continue
13        u, v, w = edge
14        if find(u) != find(v):
15            parent[find(u)] = find(v)
16            mst.append(edge)
17            total += w
18    if len(mst) == n - 1:
19        return total, mst
20    return float('inf'), []
21 def checkUniqueMST(n, edges):
22     original_weight, original_mst = kruskal_mst(n, edges)
23     for edge in original_mst:
24         new_weight, new_mst = kruskal_mst(
25             n, edges, edge
26         )
27         if new_weight == original_weight:
28             return False, new_mst, original_weight
```

```

29     return True, original_mst, original_weight
30 edges = [
31     (0, 1, 10),
32     (0, 2, 6),
33     (0, 3, 5),
34     (1, 3, 15),
35     (2, 3, 4)
36 ]
37 unique, mst, weight = checkUniqueMST(4, edges)
38 print("Is the MST unique?", unique)
39 print("MST:", mst)
40 print("Total weight:", weight)

```

Output:

```

Output

Is the MST unique? True
MST: [(2, 3, 4), (0, 3, 5), (0, 1, 10)]
Total weight: 19

=== Code Execution Successful ===

```

Time Complexity – $O(VE \log V)$ | Space Complexity – $O(V + E)$